

Agentic AI Customer 360

Atharva Pandey — IIT (BHU), Varanasi

1. Problem Approach

The goal of the system is to maintain a continuously updated Customer 360 view and use it to identify situations where a proactive intervention may be useful. Instead of sending every event directly to one LLM, I split the problem into specialist agents and a sequence of decision stages.

The final flow implemented is:

Event Stream → **Specialist Agents** → **Customer State Board** → **Synthesis** → **Life-Event Inference** → **Eligibility** → **Action** → **Critique** → **Guardrail** → **HITL** → **Execution**

The important design choice is that different coordination patterns are used for different parts of the problem. Independent signals are analysed in parallel, dependent reasoning uses sequential handoffs, and debate is activated only when there is disagreement or ambiguity. This follows the direction already established in the mid-term design.

2. Event Processing and Customer State

The system receives multiple types of customer events, mainly transaction, support, application usage and KYC/compliance events.

I keep **event time** separate from **ingestion time**. This is important because an event may arrive after the time at which it actually happened. The replay pipeline therefore checks for late events instead of assuming that events always arrive in chronological order.

In the demonstrated scenario:

- Historical events processed: **375**
- Live events processed: **116**
- Total state events: **491**
- Late events detected: **1**
- Inference checkpoints: **8**

At each checkpoint, the specialist agents analyse the new information relevant to their scope.

Specialist Agents

The specialist agents have separate responsibilities:

- **Transaction Agent:** transaction and financial activity.
- **Support Agent:** support/ticket activity and support signals.
- **Usage Agent:** digital/application usage.
- **KYC/Compliance Agent:** KYC and compliance-related signals.

The agents do not directly communicate with every other agent. Instead, they write structured findings to a **Per-Customer State Board**. This was one of the main architecture decisions from the mid-term work.

Each finding contains information such as the agent, finding type, confidence, evidence and time.

Memory

The implementation separates memory into three parts:

- **Working Memory:** current customer state and active context.
- **Episodic Memory:** customer-specific historical observations and events.
- **Semantic Memory:** reusable rules and domain-level information.

The main reason for this separation is to avoid putting the complete customer history into every agent prompt. Relevant historical information can instead be retrieved when required. This was motivated by the context-management idea studied in MemGPT, while the three-part structure itself is my adaptation for this problem.

3. Synthesis and Life-Event Inference

The Synthesis/Correlation stage combines findings from the specialist agents. The Life-Event Inference stage then looks for a higher-level customer situation instead of reacting to a single isolated event.

For example, several individually weak signals can become meaningful when considered together.

The system also supports a conditional conflict-resolution stage. If the evidence is sufficiently conflicting or ambiguous, Agent Debate and Resolution is activated. Otherwise, the pipeline continues directly.

This conditional use is intentional: running a debate stage for every checkpoint would add unnecessary computation and latency.

During the final demonstrated run, the medical-hardship scenario was resolved without requiring the debate stage because the evidence was not classified as conflicting.

LLM Failure Handling

The Life-Event stage uses an LLM when available. I also added a deterministic evidence-based fallback.

This was important during testing because an external model request can fail due to network/API availability. The system should still produce a bounded inference from the available evidence rather than crash the complete customer-processing pipeline.

This also makes the system easier to test locally.

4. From Inference to Action

After a customer state is inferred, the result is passed to the Offer/Eligibility stage.

The sequence is:

Resolved State → Offer/Eligibility → Retention/Action → Drafts → Critique

The action stage can produce an actual bounded intervention or an explicit **no-action** decision.

For the demonstrated scenario, the final result was:

- **Inferred state:** medical hardship
- **Action:** support intervention
- **Action subtype:** medical hardship payment plan

5. Round-Robin Drafting and Refinement

Instead of directly taking one generated response and executing it, the system uses multiple drafting perspectives.

In the demonstrated run, three drafts were produced:

- Customer Support Agent
- Relationship Manager Agent
- Customer Experience Agent

The drafts are then passed to the **Critique/Compliance Refiner**.

The refiner checks the candidate action for policy, compliance, risk and cost before it can move forward.

This gives the action-generation stage a clear separation between:

generate → compare/refine → validate

rather than directly executing the first generated response.

6. Guardrails and Human-in-the-Loop

A deterministic guardrail is placed before customer-facing execution.

The guardrail identifies actions that require human review. In the demonstrated scenario, the proposed customer-facing support action was routed to HITL.

The reviewer receives the important decision context:

- customer identifier
- inferred state
- confidence
- proposed action
- action subtype
- reason/evidence
- proposed message
- risk information

The human can choose:

- **Approve** – continue to execution.
- **Modify** – change the proposed action/message.
- **Reject** – block execution.

The HITL result is stored in the trace so that the final system state is not only the model decision but the complete decision path.

7. Explainability and Traceability

The system records evidence behind the inferred state.

For the demonstrated medical-hardship scenario, the evidence included historical events containing signals such as:

- “hospital”
- “pharmacy”
- “health”

The evidence is connected to source event IDs.

Therefore the decision can be followed as:

Source Events → Agent Findings → Correlated Signals → Inferred State → Action → HITL → Execution

The system also stores execution traces containing agent activity, retrievals, decisions, drafts, critique, guardrail results and HITL results.

8. Implementation and Evaluation

The system was tested end-to-end using a replayed customer scenario rather than only testing individual agents.

Metric	Observed value
Customer	CUST.00088
Historical events	375
Live events	116
Total state events	491
Late events	1
Inference checkpoints	8
Final inferred state	medical_hardship
Final action	support_intervention
Action subtype	medical_hardship_payment_plan
Final confidence	0.95
HITL status	escalated
Final execution	blocked in final checkpoint

Table 1: Observed end-to-end scenario results.

The evaluation artifact separately checks the predicted state, action and action subtype against the scenario expectation. The demonstrated final checkpoint matched the expected state, action and action subtype.

The system also generates a timestamped inferred-events file. This makes the evaluation output independent of the terminal display and gives a machine-readable artifact for scoring.

9. What Was Actually Tested

The implementation was checked at several levels.

Event pipeline: historical and live events were processed and late-event detection was observed.

Agent pipeline: Transaction, Usage, Support and KYC/Compliance findings were generated and passed into the shared customer state.

Inference: multiple inference checkpoints were executed and the life-event state was updated as new evidence arrived.

Action pipeline: eligibility, action selection, multiple drafts and critique/refinement were executed.

HITL: approve, reject and modify paths were tested. Rejection blocks execution, while approval allows execution to continue.

Observability: trace and evaluation artifacts were generated outside the terminal output.

10. Research to Implementation

The final implementation follows the main decisions from the mid-term research.

Research idea	Implementation decision
MemGPT / context management	Separate Working, Episodic and Semantic Memory with selective retrieval.
Multi-Agent Debate	Conditional Agent Debate and Resolution only when meaningful disagreement is detected.
Anthropic agent patterns	Parallel specialist analysis followed by sequential reasoning and refinement.
Blackboard architecture	Per-Customer State Board for sharing structured specialist findings.
HITL workflow	Approve / Modify / Reject checkpoint before consequential execution.
Multi-agent architecture patterns	Different coordination patterns are used where they fit the task instead of forcing one pattern over the whole pipeline.

These choices continue the research direction documented in the mid-term report. The main change from the preliminary architecture is that the final implementation now connects the architecture to an executable event replay, inference, action, guardrail, HITL and evaluation pipeline.

12. Conclusion

The main outcome of this work is an end-to-end Agentic Customer 360 prototype rather than a single LLM-based classifier.

I implemented a pipeline that takes customer events, sends them to specialist agents, maintains a shared per-customer state, retrieves relevant memory, and performs life-event inference. The inferred state is then passed through eligibility and action selection, multiple action drafts, critique/refinement, deterministic guardrails and a Human-in-the-Loop checkpoint before execution.

The implementation also handles event time and late events, produces explainability evidence and trace records, and generates an inferred-events artifact for evaluation.

In the demonstrated scenario, the system processed 375 historical events and 116 live events, detected 1 late event, and performed 8 inference checkpoints. It identified the final customer state as **medical_hardship** and selected **support_intervention** with the subtype **medical_hardship_payment_plan**. The final decision was routed through the guardrail and HITL process.

Thus, the final work connects the main parts required in the problem statement: multi-source event processing, memory, specialist agents, shared state, multi-agent coordination, proactive triggers, explainability, traceability, guardrails, HITL and evaluation.

Final Architecture

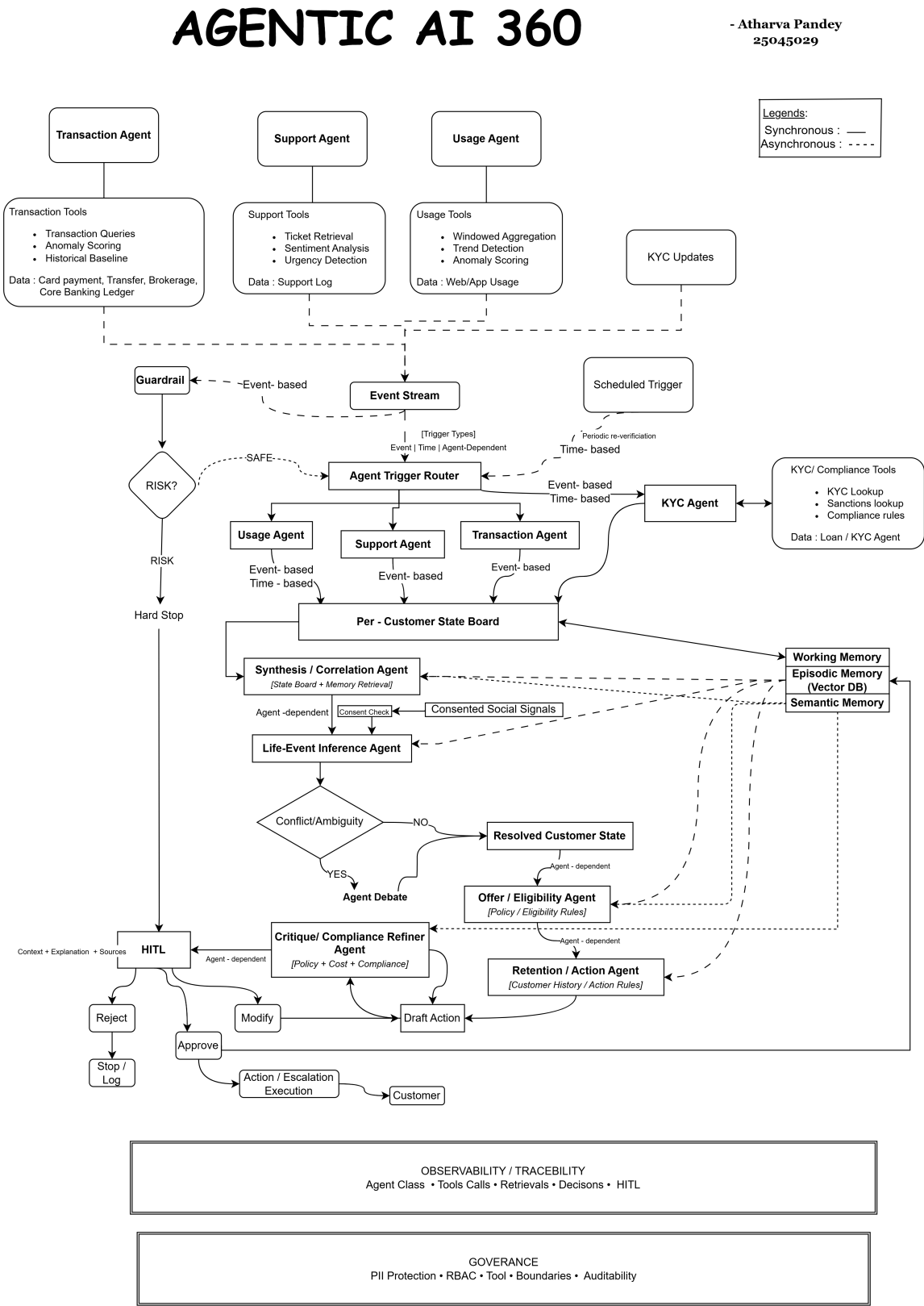


Figure 1: Final Agentic AI Customer 360 architecture.